

Rapport technique du POC de fine-tuning medical bilingue

1. Introduction

Ce document presente une synthese technique et critique du projet actuel de fine-tuning medical bilingue francais/anglais autour de [Qwen3-1.7B](#). Le POC couvre quatre volets principaux :

- la preparation d'un dataset SFT et d'un dataset DPO a partir de sources medicales publiques ;
- l'entrainement d'un adaptateur LoRA en 4-bit sur la base [unsloth/Qwen3-1.7B-unsloth-bnb-4bit](#) ;
- une evaluation comparative entre le modele de base et un checkpoint SFT ;
- une trajectoire de deploiement via une API FastAPI connectee a un serveur vLLM.

Ce rapport ne se limite pas a presenter quelques metriques. Il cherche aussi a expliquer les choix methodologiques, a situer le niveau de maturite reel du systeme, a documenter les aspects cout/performance et a exposer clairement les limites actuelles du POC.

2. Perimetre et etat actuel du projet

Le depot correspond a un POC pedagogique deja bien avance sur les volets donnees, fine-tuning et evaluation, mais encore en construction sur la partie benchmarking d'inference et sur l'industrialisation du deploiement.

Les elements actuellement disponibles dans le projet sont les suivants :

- un pipeline notebook pour l'EDA, la normalisation, la deduplication et l'anonymisation des donnees ;
- un notebook d'entrainement SFT puis DPO avec Unsloth, PEFT et TRL ;
- un notebook d'evaluation comparative sur [500](#) exemples ;
- une API FastAPI minimale avec backend [echo](#) pour les tests locaux et backend [vllm](#) pour l'inference cible ;
- un script [scripts/benchmark.py](#) prevu pour mesurer la latence et le debit en inference ;
- une base de documentation sur le deploiement Google Cloud et l'usage de vLLM.

Le workspace contient des logs exploitables pour documenter le comportement du POC. Ils permettent d'etablir un premier ordre de grandeur de latence et de debit en inference, ainsi qu'une estimation exploitable des temps d'entrainement. Cette base est suffisante pour etayer une analyse technique concrete du projet et formuler un premier cadrage cout/performance.

3. Methodologie

3.1 Preparation des donnees

Le projet assemble des sources medicales heterogenes en francais et en anglais afin de couvrir a la fois des questions ouvertes et des QCM. Les sources mentionnees dans le depot sont notamment [ANR-MALADES/MediQAI](#), [nthngdy/frenchmedmcqa](#), [keivalya/MedQuad-MedicalQnADataset](#) et [TsinghuaC3I/UltraMedical-Preference](#).

Pour un POC, le pipeline de preparation suit une logique claire et raisonnable :

1. chargement et inspection des jeux de donnees ;
2. normalisation vers un schema commun ;
3. construction des exemples SFT et DPO ;
4. deduplication des couples texte/reponse ;
5. echantillonnage par quotas ;
6. separation **train/validation/test** pour le SFT ;
7. anonymisation heuristique de certaines entites sensibles.

Cette approche presente trois avantages concrets. D'abord, elle permet de melanger des formats de supervision differents tout en conservant une interface commune. Ensuite, elle rend possible une evaluation distincte entre generation libre et QCM. Enfin, elle introduit un premier niveau de reduction du risque PII, meme si cette anonymisation reste encore insuffisante pour un contexte clinique reel.

3.2 Choix du modele de base

Le choix de **Qwen3-1.7B** ne vient pas d'une comparaison ouverte entre plusieurs modeles candidats : il etait impose par le cadre du projet. La vraie question technique est donc plus simple : est-ce un point de depart coherent pour un POC frugal, et reste-t-il defendable dans l'optique d'un passage a l'echelle sur une infrastructure raisonnable ?

Dans ce cadre contraint, les principaux arguments en faveur de ce socle restent les suivants :

- la taille **1.7B**, suffisamment compacte pour rendre le fine-tuning et l'inference envisageables sur une infrastructure GPU modeste ;
- la disponibilite d'une variante **unsloth-bnb-4bit**, qui reduit fortement la pression memoire ;
- un bon alignement avec un usage bilingue instructionnel ;
- un cout de serving bien plus faible que celui d'un modele beaucoup plus grand, a performance potentiellement moins rentable dans le cadre du POC.

En pratique, meme si le choix initial etait impose, il reste compatible avec une logique d'iterabilite experimentale, de reproductibilite et de budget maitrise. Pour un POC qui cherche ensuite a monter en charge de facon pragmatique, ce n'est peut-etre pas le meilleur modele possible, mais c'est un point de depart tout a fait defensible.

3.3 Choix des techniques d'entrainement

Le projet combine deux techniques complementaires :

- **LoRA** pour le fine-tuning supervise ;
- **DPO** pour l'alignement par preferences.

Le recours a LoRA est bien adapte a ce contexte, a la fois pour des raisons de cout et de simplicite. En evitant de mettre a jour tous les poids du modele, LoRA reduit fortement la consommation memoire et le temps d'entrainement, tout en permettant une specialisation efficace sur le domaine medical. Le chargement en 4-bit pousse encore plus loin cette logique de frugalite.

Le recours a DPO se defend egalement bien. Une fois un premier alignement supervise obtenu, DPO permet de travailler la preference entre une bonne et une moins bonne reponse sans passer par un pipeline RLHF complet, beaucoup plus lourd a mettre en oeuvre. Pour un POC, c'est une facon assez naturelle d'explorer un second niveau d'alignement clinique tout en gardant une complexite operationnelle maitrisee.

3.4 Choix du deploiement et de vLLM

Le projet retient une architecture a deux niveaux :

- une API FastAPI qui porte le contrat applicatif ;
- un serveur vLLM pour l'inference du modele.

Ce choix tient bien la route sur le plan de la performance comme sur celui de l'architecture. vLLM est bien adapte au serving LLM : il apporte un serveur OpenAI-compatible, un meilleur debit que des appels de generation plus naifs, une gestion plus efficace du KV cache via PagedAttention et un batching continu utile en cas de concurrence. FastAPI reste de son cote une bonne couche d'integration pour porter le contrat HTTP, les routes `/health` et `/generate`, les logs et d'eventuelles regles metier.

Le backend `echo` conserve dans le depot ne doit donc pas etre interprete comme un backend de production. Il sert surtout de fallback tres leger pour la CI, les tests et le packaging Docker sans GPU.

4. Configuration experimentale

4.1 Jeux de donnees cibles

Le projet decrit deux datasets finaux :

- `Maphe/medical-sft-5k` pour l'entrainement supervise ;
- `Maphe/medical-dpo-5k` pour l'alignement par preferences.

Pour le SFT, la cible documentee est de `5 000` exemples d'entrainement, `500` exemples de validation et `500` exemples de test. L'evaluation disponible porte sur le split de test de `500` exemples, avec deux sous-ensembles :

- `267` exemples de texte libre ;
- `233` exemples de QCM.

4.2 Hyperparametres documentes

Le depot documente notamment les hyperparametres suivants.

Pour LoRA :

- `r = 16`
- `lora_alpha = 16`
- `lora_dropout = 0`
- `bias = none`
- modules cibles sur les projections d'attention et de MLP

Pour le SFT :

- `2` epochs
- batch size par device `32`
- gradient accumulation `16`
- learning rate `2e-4`
- optimiseur `adamw_8bit`

- sequence length 1024

Pour le DPO :

- 1 epoch
- batch size par device 4
- gradient accumulation 8
- learning rate 5e-5
- beta = 0.1
- optimiseur adamw_8bit
- sequence length 1024

Dans l'ensemble, cette configuration est cohérente avec un objectif de fine-tuning efficace sur matériel limité. Elle favorise l'experimentation rapide, même si elle ne garantit pas à elle seule la stabilité ni la meilleure qualité clinique possible.

5. Resultats observes

5.1 Resultats quantitatifs disponibles

L'artefact de comparaison le plus complet actuellement visible dans le workspace est le resume [notebooks/eval_results/qwen3_base_vs_sft_output_summary.json](#). Il compare le modele de base au checkpoint SFT checkpoint-625 sur 500 exemples et integre, pour le texte libre, METEOR, la similarite cosinus et la distance euclidienne.

Les principaux chiffres sont les suivants :

Metrique	Modele de base	Modele fine-tune SFT	Delta
METEOR moyen texte libre	0.1361	0.1653	+0.0292
Similarite cosinus moyenne texte libre	0.3762	0.3859	+0.0097
Distance euclidienne moyenne texte libre	0.2278	0.2231	-0.0047
Score QCM first-letter	0.0515	0.4378	+0.3863
QCM corrects	12	102	+90

Pour la distance euclidienne, une baisse est une amelioration.

Ce resume fait aussi ressortir quelques tendances utiles :

- en texte libre, METEOR progresse nettement, la similarite cosinus n'augmente que legerement en moyenne et la distance euclidienne baisse legerement ;
- l'analyse par exemple reste toutefois partagee : sur 267 cas de texte libre, le fine-tune gagne 139 cas sur METEOR, seulement 113 sur la similarite cosinus contre 141 pour le modele de base, mais 159 sur la distance euclidienne contre 108 pour le modele de base ;
- en QCM, le gain est tres net et constitue le signal positif le plus robuste du POC ;
- globalement, le rang agrege reste favorable au checkpoint SFT, credite de 368 rangs 1 contre 269 pour le modele de base, avec 137 egalites.

5.2 Interpretation des resultats

L'amelioration la plus convaincante concerne les QCM. Le passage de 12/233 a 102/233 bonnes reponses montre que le fine-tuning a nettement renforce la capacite du modele a suivre un format de reponse contraint et a mieux selectionner l'information attendue. C'est un resultat utile, car une grande partie des jeux de donnees medicaux structurent les connaissances sous forme de questions a choix multiple ou de reponses attendues tres cadres.

Sur le texte libre, la lecture demande davantage de nuance. Le gain de METEOR est reel, ce qui suggere une meilleure proximite lexicale moyenne avec les references. La distance euclidienne evolue elle aussi dans le bon sens, avec une baisse de 0.2278 a 0.2231, signe d'un rapprochement moyen de la distribution lexicale vers la reference. En revanche, la similarite cosinus reste plus reservee : la moyenne progresse legerement, mais la mediane recule et le modele de base reste meilleur sur davantage d'exemples individuels. Autrement dit, l'amelioration existe, mais elle n'est pas uniforme selon la metrique retenue.

Cette dissymetrie est assez coherente avec un SFT qui apprend bien des patrons de reponse attendus, mais qui ne garantit pas encore une amelioration stable de la qualite clinique discursive. Le POC montre donc une progression reelle, surtout sur les formats structures, sans permettre pour autant de conclure a une fiabilite suffisante pour un usage sensible ni a un gain semantique uniforme sur la generation libre.

5.3 Analyse critique et nuancee

L'un des points positifs du projet est justement de ne pas s'arreter a un simple tableau de scores. Cela n'empêche pas de garder plusieurs limites importantes bien visibles.

Premiere limite : les metriques utilisees restent imparfaites pour juger une reponse medicale. METEOR, la similarite cosinus, la distance euclidienne ou le scoring par premiere lettre sont utiles pour une premiere comparaison, mais ils ne mesurent pas directement la securite, la factualite, la nuance clinique ou l'adequation a un contexte patient reel.

Deuxieme limite : l'evaluation disponible porte sur le checkpoint SFT, pas encore sur un benchmark final DPO consolide dans le depot. Il reste donc trop tot pour conclure serieusement sur l'apport effectif de DPO dans ce projet.

Troisieme limite : l'augmentation de la variance reste un signal d'alerte. Le modele fine-tune semble plus specialise, mais aussi moins regulier, y compris sur certaines metriques texte pourtant plus favorables en moyenne. Pour un POC medical, ce point compte beaucoup : quelques gains marquants ne compensent pas automatiquement plusieurs echecs critiques.

Quatrieme limite : aucun echantillon commente de reussites et d'echecs n'est encore archive dans le rapport. C'est un manque important, car une lecture clinique exige une analyse qualitative des erreurs, par exemple sur les omissions de red flags, les formulations trop affirmatives ou les reponses plausibles mais inexactes.

La conclusion la plus juste est donc la suivante : le POC montre un signal d'efficacite encourageant, surtout sur les taches structurees, mais il ne permet pas encore de revendiquer une qualite clinique robuste sur la generation libre.

6. Analyse cout/performance

6.1 Performance d'inference : latence et debit

Le projet contient deja un dispositif de mesure dans `scripts/benchmark.py`, prevu pour benchmarker la latence et le debit du backend `vllm` a travers l'API FastAPI. Le script collecte notamment :

- la latence par requete en millisecondes ;
- la latence moyenne, mediane et p95 ;
- le nombre de tokens generes ;
- le debit en tokens par seconde.

Ce choix de mesure est pertinent, car il observe la performance au niveau applicatif reel et pas seulement au niveau du moteur vLLM. Il integre donc le cout de la couche HTTP et donne une vision plus proche d'un service effectivement deploye.

Les logs actuellement disponibles ne couvrent qu'un faible nombre de requetes completes. Les valeurs ci-dessous doivent donc etre lues comme un ordre de grandeur pour une campagne sequentielle d'environ 50 requetes realisee dans des conditions comparables : prompt medical de longueur proche, `max_new_tokens = 2048` et `temperature = 0.3`.

Metrique	Moyenne	Mediane	p95
Latence API de bout en bout	14,6 s	14,1 s	15,9 s
Latence generation vLLM	14,5 s	14,1 s	15,7 s
Debit de generation	56,6 tok/s	56,6 tok/s	58,4 tok/s
Tokens generes par reponse	~820	~794	~919

Le debit moyen pondere peut etre retenu autour de 56,6 tok/s. L'ecart entre la latence de generation et la latence API totale reste faible, ce qui suggere que le surcout de la couche HTTP/FastAPI reste secondaire devant le temps de generation.

Attention cependant :

- les requetes restent sequentielles, donc ces chiffres ne disent rien sur le comportement sous concurrence.

En pratique, pour le modele `Qwen3-1.7B` servi par `vLLM`, l'ordre de grandeur a retenir est d'environ 14 a 16 secondes par reponse longue, pour un debit voisin de 56 a 58 tok/s en generation sequentielle. Sur cette base, une serie de 50 generations longues traitees l'une apres l'autre représenterait environ 12 a 13 minutes de calcul.

6.2 Cout d'infrastructure pour le deployment

La documentation GCP du projet recommande une VM `n1-standard-4` avec 1 x `NVIDIA T4`, idealement en mode Spot. Le cout indicatif documente dans le depot est de l'ordre de 0,19 \$ a 0,26 \$ / heure pour cette configuration frugale.

Sur cette base, on peut etablir un cadrage simple du cout de serving GPU :

Duree d'utilisation GPU	Cout estime bas	Cout estime haut
1 heure	0,19 \$	0,26 \$

Duree d'utilisation GPU	Cout estime bas	Cout estime haut
10 heures	1,90 \$	2,60 \$
30 heures	5,70 \$	7,80 \$
100 heures	19,00 \$	26,00 \$

Il faut naturellement ajouter a cela le stockage persistant et, selon l'architecture retenue, le cout d'une petite instance CPU ou de quelques services reseau. Le projet rappelle d'ailleurs une bonne pratique tres simple : arreter la VM hors utilisation pour que la facturation du CPU et du GPU s'interrompe immediatement.

Cette strategie convient bien a un POC, avec un corollaire assez clair : tant que le service n'est pas automatise et benchmarké, le cout reste surtout pilote par le temps de machine allumee, et non encore par une capacite verifiee en requetes par seconde.

6.3 Cout d'infrastructure pour l'entrainement

Le besoin exprime porte aussi sur les heures GPU d'entrainement. Sur ce point, les resultats engranges permettent une estimation partielle, sans permettre une reconstitution exacte de la duree totale d'entrainement.

Ils montrent au minimum que le pipeline SFT puis DPO a bien ete execute, et que la phase DPO dispose d'une duree explicite de 3029 s, soit environ 0,84 heure GPU sur 1 GPU. Pour le SFT, la duree moyenne retenue ici est d'environ 13 min, soit 0,22 heure GPU.

Sur cette base, l'entrainement du modele fine-tune present dans ce depot peut etre resume ainsi :

Etape	Source	Heures GPU
SFT LoRA	moyenne observee	0,22 h
DPO	mesure observee dans le notebook (3029 s)	0,84 h
Total POC actuel	somme SFT + DPO	1,06 h

Cet ordre de grandeur confirme surtout le point central du POC : pour un modele 1.7B adapte avec LoRA et quantification 4-bit, le budget d'entrainement reste compatible avec une logique d'experimentation frugale sur GPU unique.

Si l'on veut malgre tout traduire ces heures GPU en cout purement indicatif avec la meme hypothese lineaire que pour la VM GPU documentee plus haut (0,19 \$ a 0,26 \$ / heure), on obtient un ordre de grandeur tres bas :

Heures GPU cumulees	Cout estime bas	Cout estime haut
1,06 h	0,20 \$	0,28 \$

Il faut toutefois lire ce tableau avec prudence, car le tarif de serving T4 documente pour GCP ne correspond pas necessairement au materiel effectivement utilise pendant l'entrainement.

6.4 Lecture cout/performance globale

Au stade actuel, le message technique le plus utile a retenir est le suivant :

- du cote qualite, le POC montre un gain tangible sur les taches structurees ;
- du cote cout, l'architecture choisie est volontairement frugale et compatible avec un budget etudiant ;
- du cote performance d'inference, un premier ordre de grandeur est maintenant documente autour de **14 a 16 s** par reponse longue et **56 tok/s** en sequentiel ;
- du cote entrainement, le DPO est mesure a **0,84** heure GPU et le pipeline SFT + DPO documente ici represente environ **1,06** heure GPU observee, avec une marge d'incertitude liee au caractere partiellement archive des runs.

En resume, la trajectoire cout/performance du projet parait saine : le modele actuel reste peu couteux a adapter, et le serving observe sur GPU unique est exploitable pour un POC. Elle n'est toutefois pas encore completement demontree par un benchmark de charge archive, multi-prompts et multi-concurrence.

7. Comment passer a l'echelle vers un modele de classe Qwen 32B

Si l'objectif devient de dépasser un POC frugal pour viser une meilleure qualite de generation, une piste naturelle consiste a conserver l'architecture applicative actuelle tout en remplaçant le moteur de generation par un modele beaucoup plus grand, par exemple un Qwen de classe **32B**. L'interet d'un tel changement serait d'augmenter la capacite de raisonnement, la richesse des reformulations, la robustesse multilingue et la tenue des reponses longues. En contrepartie, le projet changerait de categorie technique : on ne parlerait plus d'un serving leger sur GPU unique, mais d'un systeme d'inference distribue et nettement plus couteux.

L'idée la plus importante est donc la suivante : le passage a l'echelle ne suppose pas forcément de jeter l'architecture actuelle. La separation deja presente entre FastAPI et le backend d'inference constitue au contraire une bonne base. L'API peut continuer a porter le contrat **/health** et **/generate**, la validation des entrees, les logs et d'éventuels garde-fous metier, tandis que la couche inferieure evolue d'un backend **echo** ou **vllm** simple vers un serving distribue capable de charger un modele de tres grande taille.

7.1 Ce qui change techniquement avec un modele 32B

Le saut entre **1.7B** et **32B** n'est pas lineaire du point de vue operationnel. Il change simultanément :

- la quantite de VRAM necessaire pour charger les poids du modele ;
- la bande passante memoire requise pour maintenir une latence acceptable ;
- le nombre de GPU necessaires pour l'inference ;
- la complexite du deploiement reseau, du scheduling et de l'observabilite ;
- le cout unitaire par requete si aucune optimisation n'est mise en place.

Concretement, un modele de cette classe ne se deploie plus de facon realiste sur une simple **T4**. Meme en quantification aggressive, il faut raisonner en termes de plusieurs GPU haut de gamme, avec parallelisme tensoriel et parfois pipeline parallelism selon le moteur retenu et la taille effective du contexte. Autrement dit, le changement de modele implique un changement d'infrastructure bien avant d'impliquer un changement de code applicatif.

7.2 Architecture cible pour le serving a grande echelle

Dans une trajectoire de passage a l'echelle serieuse, l'architecture la plus defendable resterait une architecture en couches :


```
Client -> FastAPI -> service d'inference distribue -> cluster GPU -> modele Qwen 32B
```

FastAPI conserverait plusieurs responsabilites utiles :

- stabiliser le contrat HTTP pour les clients ;
- appliquer des validations de payload et des bornes de securite ;
- journaliser les requetes, les latences et les erreurs ;
- ajouter des politiques metier, par exemple sur les demandes urgentes ou hors perimetre ;
- masquer les details d'implementation du moteur de serving.

La couche d'inference, en revanche, devrait monter en gamme. Dans cette hypothese, **vLLM** reste un candidat credible, mais il faudrait l'exploiter dans un mode distribue adapte aux grands modeles, avec :

- parallelisme tensoriel multi-GPU ;
- quantification si elle reste compatible avec la qualite attendue ;
- gestion stricte du KV cache et des longueurs de contexte ;
- supervision des temps de reponse, de la saturation GPU et des erreurs de generation.

L'API actuelle n'aurait alors pas besoin d'etre refondue en profondeur. Le vrai travail porterait surtout sur le backend de serving, l'infrastructure GPU et l'observabilite de production.

7.3 Strategie de migration recommandee

Le point critique n'est pas seulement de choisir un plus gros modele, mais de le faire sans perdre la reproductibilite acquise sur le POC. Une trajectoire raisonnable pourrait se decomposer en quatre paliers.

1. conserver le contrat FastAPI actuel et remplacer d'abord le backend **echo** par un backend **vllm** reel sur un modele intermediaire ;
2. valider le benchmark de bout en bout sur une seule machine GPU plus solide que la **T4** afin d'etablir une premiere base de latence, debit et VRAM ;
3. migrer ensuite vers un modele plus grand necessitant plusieurs GPU, avec parallelisme explicite et mesures de charge ;
4. seulement apres cette stabilisation, envisager un modele de classe **32B** avec objectifs de SLA, budget et politiques de routage clairement definis.

Cette progression est importante, car elle evite de passer brutalement d'un POC peu couteux a une pile tres lourde sans zone intermediaire d'apprentissage. Dans beaucoup de projets, la bonne decision n'est pas de passer directement au plus gros modele possible, mais de verifier a partir de quel niveau de taille le gain qualitatif justifie reellement la hausse de complexite et de cout.

7.4 Impact sur le fine-tuning et l'alignement

Le passage à un Qwen **32B** pose aussi une question stratégique sur l'entraînement. Sur un petit modèle, un fine-tuning LoRA en 4-bit reste compatible avec une logique de frugalité. Sur un modèle beaucoup plus grand, même un adaptateur LoRA devient plus exigeant en ressources, en stockage intermédiaire, en temps de synchronisation et en orchestration. Le coût d'expérimentation augmente donc fortement, même si l'on évite toujours un full fine-tuning.

Il faut toutefois rester prudent sur la façon d'extrapoler ce coût. Dans un fine-tuning **LoRA**, on n'actualise qu'une petite fraction des poids, ce qui réduit fortement le coût mémoire et l'état d'optimisation. En revanche, le modèle complet continue à être traversé en avant et en arrière à chaque étape. Il serait donc trop simpliste de projeter les heures GPU du **32B** en multipliant mécaniquement le temps observé sur **1.7B** par le seul ratio de taille entre les deux modèles.

Pour le POC actuel, les temps observés peuvent déjà être distingués entre la phase de fine-tuning supervisé et la phase d'alignement par préférences :

Étape	Temps observé	Part du total
SFT LoRA	0,22 h GPU	20,8 %
DPO	0,84 h GPU	79,2 %
Total pipeline actuel	1,06 h GPU	100 %

Cette séparation est utile, car elle montre que, dans l'état actuel du projet, le coût d'entraînement n'est pas réparti uniformément : la phase **DPO** représente la majeure partie du budget GPU observé. Autrement dit, si l'on cherche à contenir les coûts dans une future montée en gamme, ce n'est pas seulement la taille du modèle qui compte, mais aussi la place effective donnée au pipeline d'alignement.

On peut malgré tout donner un ordre de grandeur prudent pour un passage à **32B**. Le ratio de taille entre **32B** et **1.7B** est d'environ **18,8x**. En appliquant ce ratio de façon volontairement simple aux deux composantes du pipeline actuel, on obtient la projection indicative suivante :

Étape	Base observée 1.7B	Projection simple 32B
SFT LoRA	0,22 h GPU	4,1 h GPU
DPO	0,84 h GPU	15,8 h GPU
Total	1,06 h GPU	19,9 h GPU

Cette projection reste volontairement simplifiée. En pratique, un **32B** imposerait probablement une batch size plus contrainte, davantage de synchronisation, plus de pression mémoire et une infrastructure plus lourde. Pour cette raison, un cadrage plus réaliste pour un premier budget exploratoire serait plutôt de l'ordre de **20 à 30** heures GPU pour reproduire un pipeline comparable en **LoRA**, à jeu de données et nombre d'epochs similaires.

En se basant sur les tarifs observés sur RunPod, un **A100 80GB** se situe autour de **1,19 à 1,39 \$/h**, tandis qu'un **H100 80GB** se situe plutôt autour de **1,99 à 2,69 \$/h**. Cela permet de transformer plus concrètement l'ordre de grandeur en budget calculatoire.

GPU RunPod	Prix horaire	Coût pour 20 h GPU	Coût pour 30 h GPU
A100 80GB PCIe	1,19 \$/h	23,8 \$	35,7 \$
A100 80GB SXM	1,39 \$/h	27,8 \$	41,7 \$
H100 80GB PCIe	1,99 \$/h	39,8 \$	59,7 \$
H100 80GB SXM	2,69 \$/h	53,8 \$	80,7 \$

Ces ordres de grandeur montrent que la phase purement calculatoire d'un premier fine-tuning **LoRA** sur un **32B** reste encore accessible financièrement. En première approximation, un run de reproduction du pipeline actuel se situerait donc autour de **24 à 81 \$** selon le type de GPU retenu. Dans une logique de frugalité, l'**A100** apparaît comme le point d'équilibre le plus crédible entre coût, capacité mémoire et simplicité d'accès.

Il faut toutefois ajouter à cette base le coût des essais invalidés, des variations d'hyperparamètres, des évaluations intermédiaires, ainsi que les éventuels surcoûts de stockage et d'orchestration. En pratique, pour une première campagne sérieuse avec plusieurs essais **SFT + DPO**, une enveloppe de planification plus réaliste serait plutôt de l'ordre de **100 à 300 \$**, voire davantage si plusieurs runs comparatifs ou plusieurs configurations de contexte doivent être testés.

Cette estimation doit toutefois être lue comme un ordre de grandeur de planification, pas comme une prédiction fiable de durée murale. En pratique, le temps horloge pourrait être réduit avec plusieurs GPU plus puissants, mais le total en heures GPU resterait du même ordre ou augmenterait légèrement à cause des surcoûts de parallélisme.

La conclusion la plus défendable, à ce stade, est donc la suivante : un **32B** reste envisageable avec **LoRA**, mais il ferait clairement changer le projet d'échelle opérationnelle. On sortirait du régime très frugal du POC actuel sur infrastructure légère pour entrer dans un cadre demandant au minimum un GPU haut de gamme avec davantage de VRAM, et souvent plusieurs GPU selon la précision retenue, la longueur de contexte, la batch size et la contrainte de délai.

L'enjeu n'est donc pas une impossibilité absolue, mais plutôt une perte de simplicité expérimentale. À partir de cette taille de modèle, il devient nécessaire de mesurer empiriquement les temps d'entraînement, les coûts réels d'itération et les contraintes de stabilité avant d'annoncer un budget plus ferme.

Dans ce contexte, deux options paraissent plus réalistes qu'un simple portage du POC actuel :

- soit utiliser le grand modèle principalement en inférence, sans fine-tuning immédiat, afin d'évaluer d'abord le gain qualitatif brut ;
- soit réserver le fine-tuning à des cas d'usage très ciblés, avec **LoRA**, jeu de données mieux nettoyé, protocole d'évaluation plus strict et infrastructure adaptée.

Cette distinction est importante pour la suite du projet. Si le besoin principal est d'améliorer fortement la qualité de réponse, l'inférence sur un grand modèle peut suffire dans un premier temps. Si l'objectif est en plus de spécialiser finement le comportement médical, alors il faut prévoir une véritable feuille de route **MLOps** couvrant les datasets, l'alignement, le versioning des adapters, la traçabilité des runs et les campagnes d'évaluation.

7.5 Arbitrage cout, performance et valeur metier

Le principal frein a un passage a l'echelle vers **32B** ne sera probablement pas la faisabilite logicielle, mais l'economie du systeme. A ce niveau, le projet doit raisonner non plus seulement en heures GPU, mais en cout par requete utile, en latence acceptable pour l'utilisateur, en debit sous concurrence et en gain clinique reel par rapport a un modele plus petit.

Autrement dit, la bonne question n'est pas seulement : peut-on servir un Qwen **32B** ? La bonne question est plutot : dans quelles conditions ce surcout se traduit-il par une amelioration suffisamment nette pour justifier l'infrastructure, les risques operationnels et l'effort de maintenance ?

Pour un projet medical, cette question est encore plus sensible. Un plus gros modele peut produire des reponses plus fluides et plus convaincantes, sans pour autant garantir a lui seul la surete clinique. Le passage a l'echelle doit donc etre pense comme un changement d'architecture globale : moteur plus puissant, certes, mais aussi evaluation plus exigeante, observabilite plus fine, garde-fous applicatifs plus stricts et gouvernance plus mature sur les usages autorises.

En ce sens, l'architecture actuelle du projet joue deja un role utile : elle fournit un premier squelette separant clairement la couche applicative de la couche d'inference. Si le POC devait evoluer vers une offre plus ambitieuse, cette separation permettrait justement de faire grandir le moteur de generation jusqu'a une classe **32B** sans devoir redefinir entierement l'interface exposee aux utilisateurs.